

Grafika Komputerowa

Zadanie 2: Eliminacja elementów zasłoniętych — konspekt

Adam Boros, Kacper Warpechowski

Cel zadania

Implementacja algorytmu BSP tree (Binary Space Partitioning) do poprawnego rysowania zbioru trójkątów w przestrzeni 3D z uwzględnieniem wzajemnego zasłaniania. Scena wczytywana z pliku tekstowego, obiekty wypełnione jednolitym kolorem.

Realizacja

Projekt napisany w Pythonie; pygame do obsługi okna i rysowania wypełnionych wielokątów, numpy wyłącznie do mnożenia macierzy.

Scena: kilka trójkątów 3D wczytanych z pliku tekstowego, każdy opisany współrzędnymi trzech wierzchołków (x, y, z) i kolorem. Trójkąty są wypukłe z definicji. Zgodnie z założeniami zadania żadne trzy trójkąty nie tworzą cyklu wzajemnego zasłaniania. Rzutowanie perspektywiczne zrealizowane macierzą P analogicznie jak w zadaniu 1.

Algorytm BSP tree:

1. **Budowa drzewa:** wybierz dowolny trójkąt jako węzeł. Jego płaszczyzna dzieli przestrzeń na dwie półprzestrzenie. Trójkąty po stronie kamery trafiają do lewego poddrzewa, pozostałe do prawego. Powtarzaj rekurencyjnie aż do wyczerpania listy trójkątów.
2. **Rysowanie (back to front):** przejdź drzewo rekurencyjnie — rysuj najpierw poddrzewo po stronie przeciwnej do kamery (dalsze), potem węzeł, potem poddrzewo po stronie kamery (bliższe). Każdy bliższy trójkąt nadmalowuje dalsze, uzyskując poprawny efekt zasłaniania.

Kolejność rysowania wynika ze struktury drzewa i aktualnej pozycji kamery.

Format pliku wejściowego

Każdy wiersz opisuje jeden trójkąt: $x_1\ y_1\ z_1\ x_2\ y_2\ z_2\ x_3\ y_3\ z_3\ R\ G\ B$.

Testy poprawności

- Trójkąt bliższy kamerze powinien całkowicie przykrywać dalszy w miejscu nakładania — weryfikacja wizualna dla sceny z trójkątami ustawionymi jeden za drugim wzdłuż osi Z .
- Zamiana kolejności wierszy w pliku wejściowym nie powinna zmieniać wyniku — drzewo BSP jest budowane od nowa przy każdym uruchomieniu i kolejność rysowania wynika ze struktury drzewa, nie z kolejności wczytania.